

UNIFEOB
CENTRO UNIVERSITÁRIO DA FUNDAÇÃO DE ENSINO
OCTÁVIO BASTOS
ESCOLA DE NEGÓCIOS

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
CIÊNCIAS DA COMPUTAÇÃO

Computação em Nuvem

ESTUDANTES:

25000151 - Guilherme Portilho da Rosa Santi

25001176 - Henrique de Oliveira Molinari

25000019 - João Augusto de Freitas

25000795 - Kauan Leander Leandrini

São João da Boa Vista–SP

2026

Configuração do Ambiente e Criação da Estrutura da API

Disciplina: Projeto Integrador (PI 2026.2)

Atividade do dia: 01/09/2026 — Prof. Nivaldo

Projeto: SOUFER Tools — controle de retirada e devolução de ferramentas do almoxarifado da Soufer

Aluno: João Augusto de Freitas

Repositório: github.com/joaoaugusto-dev/PI-2026.2

1. Configuração do ambiente de desenvolvimento

A API do SOUFER Tools roda em Node.js 20+ com TypeScript, Express para as rotas e o driver `pg` (node-postgres) falando direto com um PostgreSQL próprio — a equipe decidiu não depender de BaaS (Supabase ou parecido) pra manter controle total sobre autenticação e infraestrutura.

Passos pra colocar o ambiente local de pé:

1. `npm install` dentro de `/api`.
2. Cópia do `.env.example` para `.env`, com as credenciais do PostgreSQL local (host, porta, nome do banco, usuário, senha) e os segredos de JWT — chave secreta, expiração do token de almoxarife (8h) e do token de consulta do quiosque (15min).
3. Criação do banco `soufer_dev` no PostgreSQL local instalado na máquina (versão 14).

Teve uma pegadinha no meio do caminho: o usuário `postgres` do Postgres local não tinha senha definida. O `psql` direto na máquina conecta por peer auth (sem senha), mas o driver `pg` da API conecta via TCP, que exige senha — e a conexão ficava caindo com "password authentication failed". Resolvido com `ALTER ROLE postgres WITH PASSWORD 'postgres'`, batendo com o que o `.env.example` já esperava.

2. Estrutura base do projeto e configuração do banco de dados

A estrutura da API (Express, camadas de controller/service/middleware/validator, Swagger) já tinha sido levantada numa etapa anterior do projeto. O trabalho de hoje foi validar se essa estrutura sobe do zero num ambiente novo — e não subiu de primeira.

```
api/
├─ db/
│  ├─ migrations/0001_init.sql  # 13 tabelas, enums, triggers, views
│  └─ seed.sql                 # dados de teste
├─ scripts/
│  ├─ migrate.ts
│  └─ seed.ts
└─ src/
   ├─ app.ts / server.ts
   ├─ config/          (env, database, swagger)
   ├─ controllers/     (health, auth)
   ├─ services/        (auth)
   ├─ middlewares/     (auth, authorize, errorHandler, logger, validate)
   ├─ routes/v1/       (health, auth, consulta)
   └─ validators/
```

Rodando `npm run db:migrate`, a migration `0001_init.sql` criou sem erro as 13 tabelas do domínio (setores, grupos e subgrupos de ferramentas, atividades, usuários, colaboradores, ferramentas, itens de kit, empréstimos, ocorrências, notificações, feriados e auditoria), os enums de status e as triggers — inclusive a que gera automaticamente o código de identificação de 4 dígitos de cada ferramenta.

O `npm run db:seed`, porém, quebrou na primeira tentativa: o script tentava inserir numa tabela `categorias` que não existe mais — ela foi renomeada pra `grupos_ferramentas` numa revisão de schema feita depois de uma visita técnica ao almoxarifado da Soufer, e o `seed.sql` não tinha acompanhado essa mudança. A mesma revisão também removeu as colunas `codigo_cracha` e `cargo` de `colaboradores`, e trocou `codigo_patrimonio` por `codigo_identificacao` em `ferramentas`. Corrigi o seed pra essas colunas atuais.

Ainda assim quebrou de novo, agora com violação de chave estrangeira em `colaboradores.setor_id`. O script original usava IDs fixos (`setor_id = 1, 2 ...`), só que a sequência `SERIAL` do Postgres não é transacional: a primeira tentativa de seed, mesmo com rollback, já tinha avançado a sequência de `setores`, e os IDs fixos deixaram de bater. Troquei os IDs fixos por subconsulta pelo nome do setor/grupo — assim o seed continua funcionando mesmo depois de uma tentativa que falhou no meio.

3. Definição das rotas iniciais

Rotas expostas em `/v1` neste momento do projeto:

Método	Rota	Função
GET	/v1/health	Healthcheck da API + teste de conexão com o PostgreSQL
POST	/v1/auth/login	Login do almoxarife (e-mail + senha, token JWT de 8h)
GET	/v1/auth/me	Dados do usuário autenticado a partir do token
POST	/v1/consulta/sessao	Sessão de consulta do quiosque (matrícula, token de 15min, sem senha)

Todas seguem o envelope de resposta padrão do projeto — `{ data, meta }` no sucesso e `{ error: { code, message, details } }` no erro — e qualquer rota fora desse conjunto cai num 404 padronizado.

4. Implementação e teste da conexão com o banco de dados

A conexão com o PostgreSQL é feita por um pool (`pg.Pool`) configurado em `src/config/database.ts`, com um helper `testConnection()` que roda um `SELECT NOW()` simples pra confirmar que o banco está respondendo. A rota `GET /v1/health` usa esse helper e devolve 200 com `database.status: "connected"` quando está tudo certo, ou 503 se o banco cair.

Com o servidor rodando (`npm run dev`), o healthcheck confirmou a conexão real com o banco:

```
GET /v1/health -> 200

{
  "data": {
    "status": "ok",
    "database": {
      "status": "connected",
      "name": "soufer_dev",
      "serverTime": "2026-09-05T13:12:56.055Z"
    }
  }
}
```

5. Testes das rotas com Insomnia

Os testes foram feitos no Insomnia, cobrindo cenário de sucesso e de erro em cada rota que já tem lógica de negócio implementada:

Rota	Cenário	Resultado
POST /v1/auth/login	credenciais corretas	✓ 200, token emitido
POST /v1/auth/login	senha errada	✗ 401 INVALID_CREDENTIALS
GET /v1/auth/me	com token válido	✓ 200, dados do usuário
GET /v1/auth/me	sem token	✗ 401 TOKEN_NOT_PROVIDED
POST /v1/consulta/sessao	matrícula existente	✓ 200, token de 15min
POST /v1/consulta/sessao	matrícula inexistente	✗ 404 COLABORADOR_NOT_FOUND

O último caso (`consulta/sessao`) só passou a funcionar depois da correção descrita na seção 2: o serviço de autenticação ainda consultava a coluna `codigo_cracha` , removida do banco. Antes do ajuste, qualquer chamada pra essa rota quebrava com erro 500 (coluna inexistente), independente da matrícula informada. A busca por matrícula está funcionando; a busca por número de crachá fica pendente até a equipe decidir se esse identificador volta pro banco — está anotado como item em revisão no guia do projeto.

A coleção usada nesses testes foi exportada e versionada no repositório em `api/docs/insomnia-collection.json` , já disponível pra quem for continuar o CRUD de ferramentas nas próximas atividades.

Resultado

Ambiente local funcional — Postgres e API rodando, dados de teste populados — com as 4 rotas atuais de `/v1` testadas e validadas via Insomnia. No processo, dois bugs de schema foram encontrados e corrigidos (seed desatualizado e consulta a coluna removida no serviço de autenticação), que teriam travado qualquer atividade futura sobre esse banco. O código está na branch `chore/nivaldo-01-09-setup-ambiente-api` do repositório do projeto.